

AdaGrad

⇒ AdaGrad (Adaptive Gradient)

⇒ Key Idea ⇒ parameters with large gradients should have lesser LR while the parameters with small gradients should have the LR increased.

⇒ Key Innovation →

→ accumulates squared gradients

→ if accumulated gradients are large
→ LR becomes smaller

→ if accumulated gradients are small
→ LR becomes larger

⇒ ideal for -

→ sparse datasets

→ features with diff. frequencies

→ multiple NLP tasks

⇒ It automatically tries to balance the learning process. If the set of parameters are updated frequently, the smaller effective LR is used. If the params are updated infrequently, then large LR.

Ex. For a word embedding model
 "the" appears many times $\Rightarrow$ large gradients accumulate $\Rightarrow$ LR (as already learnt well)

"oak" small appears rarely $\Rightarrow$ gradients accumulate $\Rightarrow$ LR (needs more learning)

$\Rightarrow$ Algo. -

init $\theta_0, \eta, \epsilon, G_0 = 0$ $\xrightarrow{\text{gradient accumulator}}$

for $i = 1$ to T

$$g_t = \nabla_{\theta} J(\theta)$$

$\rightarrow$ computing gradients

$$G_t = G_{t-1} + g_t \odot g_t$$

$\rightarrow$ accumulates sq. grads.

element-wise multiplication

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t} + \epsilon} \odot g_t$$

$\Rightarrow$ updating the params

$\rightarrow$ helps to prevent effective div. by 0.
 LR (decreases over time)

- For any parameter θ_i at time t

$$G_{t,i} = \sum_{\tau=1}^t (g_{\tau,i})^2$$

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,i}} + \epsilon} \cdot g_{t,i}$$

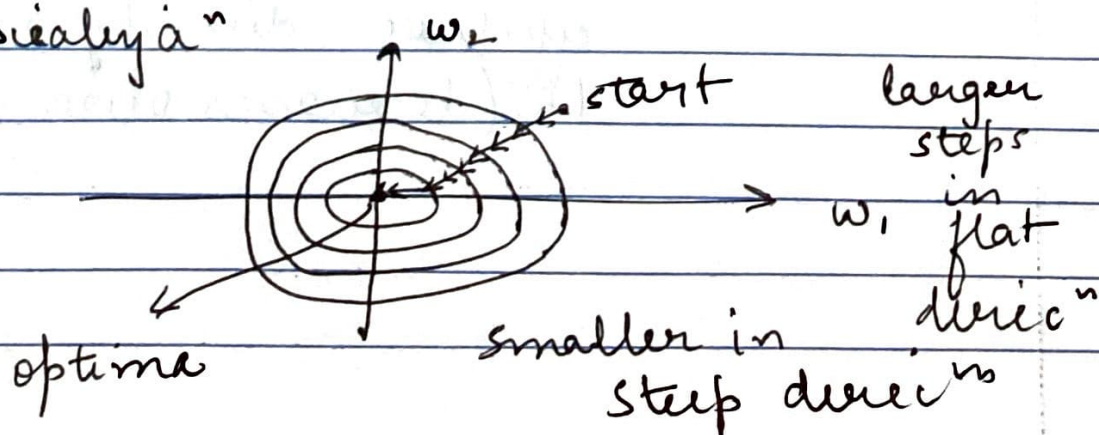
⇒ Some observations

1. LR decreases monotonically
2. early gradients will have a long-term lasting effect
3. each param. will have its own LR
4. G_t tends to grow up bounding

⇒ Advantages

1. no tuning LR manually
2. automatic per-parameter adaptaⁿ
3. excellent for sparse features
4. convergence guaranteed theoretically

⇒ Visualization



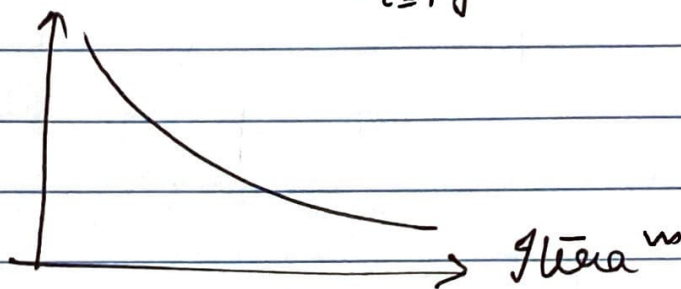
- automatically scaled LR for each dim.
- reduces oscillations in steep direc^{ns}
- maintains a good progress in flat direc^{ns}

⇒ Limita^{ns} of Adagrad

since $G_t = \sum_{\tau=1}^t g_{\tau}^2$ } accumulated gradient
grows monotonically

leads to aggressive LR decay

$$\lim_{t \rightarrow \infty} \underbrace{\frac{\eta}{\sqrt{G_t} + \epsilon}}_{\text{LR}} = \lim_{t \rightarrow \infty} \frac{\eta}{\sqrt{\sum_{\tau=1}^t g_{\tau}^2} + \epsilon} \Rightarrow 0$$



- at one pt. LR becomes too small
- training effectively stops
- difficult to escape the local minima, if at later steps in training.